

DATA PROTECTION & ENCRYPTION AT REST

Security engineering dossier / 01 — Implemented mechanisms

I am Amin Azimi, AI Architect and developer of Aegis Agent. I document the actual protection mechanisms in my codebase and explicitly separate data encoding, message authentication, access scoping and encryption at rest. I do not treat a security-themed class name as evidence of an independently verified security guarantee.

01 | The data surfaces I distinguish

My implementation includes a process-local TenantMemoryVault, persistent SQLite state for task records and cognitive-twin profiles, local generated files and tenant backup archives. These surfaces have different lifetimes and protection characteristics; they cannot be described as one uniformly encrypted storage layer.

02 | Tenant memory value transformation

I serialize memory values as JSON, XOR their bytes against a repeating SHA-256-derived key and encode the result using URL-safe Base64. I describe this accurately as reversible obfuscation, not authenticated encryption. A source-code fallback key exists, and the transform provides no cryptographic integrity tag.

03 | SQLite persistence is not disk encryption

I use SQLite with WAL journal mode and foreign-key enforcement. WAL is a database journaling mechanism, not encryption at rest. The inspected persistence layer does not configure SQLCipher or another database-encryption provider. I therefore do not claim that all on-disk SQLite records are encrypted.

04 | Signed capability tokens

I use HMAC-SHA256 to authenticate canonical JSON claims in scoped tool tokens. The encoded token carries tool, tenant, task, capability and expiration fields. Signing protects against undetected claim modification when the secret remains private; Base64 does not conceal token contents. The source also contains a fallback signing secret.



PROTECTION BOUNDARIES

05 | Backup and exported artifacts

I provide a tenant-targeted local backup function that places selected SQLite rows and receipt files into a compressed ZIP archive. ZIP compression is not encryption. The inspected backup path does not establish archive-level encryption or automatic secure erasure of prior copies.

06 | Separation is not authorization

I distinguish tenant-keyed storage from authenticated tenant ownership. In-memory namespaces and tenant-filtered database queries help organize and scope records, but callers must still be bound to trusted identities. I do not claim end-to-end multi-tenant authorization solely from a tenant_id argument.

07 | Key rotation and deletion semantics

I expose a vault master-key rotation setter, but it does not re-encrypt existing entries. I therefore do not characterize rotation as a complete cryptographic migration. Likewise, deleting an in-memory value or a generated listing is not proof that every backup, database record or external copy has been erased.

08 | Engineering priorities for hardening

My next security-hardening targets are authenticated encryption for sensitive values, secrets provisioned outside source code, safe key migration, encrypted backup handling, authenticated tenant binding, explicit data-retention rules and independently executed negative security tests. These are engineering priorities, not claims of features already delivered.

09 | Evidence and publication standard

This dossier is grounded in direct inspection of core/tenant_memory.py, core/persistence.py, core/tool_tokens.py and core/twin_backup.py. I have not represented code inspection as a penetration test, regulatory audit or verified production deployment.

SECURITY STATUS — ACTIVE DEVELOPMENT
I treat encryption at rest as a hardening requirement, not as a completed blanket guarantee. I will only describe a stronger protection property once its implementation and validation evidence support it.